ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY & SCIENCES

(AUTONOMOUS) | VISAKHAPATNAM

UNIT V – CASE STUDY DOCUMENTATION

STUDIA

Student Management System

MongoDB, Database Operations and Cloud Deployment

Course Details	Information
Course Title	Full-Stack Web Development
Course Code	23CM4121
Unit	Unit V – MongoDB and Deploying Applications
Student Name	Mohammed Almas
Registration Number	A24126552098
Academic Year / Semester	3rd Year – 1st Semester
Faculty / Guide	P. Santosh Kumar

Academic Year 2026–27

CONTENTS

Section	Title
PART A	Comprehensive Question and Problem Statement
PART B – 1	Requirement Analysis and System Design
PART B – 2	MongoDB Document Model and Database Operations
PART B – 3	Backend and Frontend Implementation
PART B – 4	Testing, Debugging and Validation
PART B – 5	Cloud Deployment and Security
PART C	Output Screenshots and Evidence
Conclusion	Outcome, Limitations and Future Enhancements

Project Snapshot

Layer	Technology / Platform	Role
Frontend	React, Vite, Axios, CSS, Lucide React	User interface and student workflows
Backend	Node.js, Express.js, Mongoose	REST API and server-side data operations
Database	MongoDB Atlas	Persistent student records
Backend hosting	Render	Hosts the Express API
Frontend hosting	Vercel	Builds and serves the React application

Live application: https://studia-frontend.vercel.app

Backend service: https://studia-backend-orll.onrender.com

Source repository: https://github.com/mohammedalmas-csm98/FS-LAB

Project folder in repository: CASE STUDY/Student-Management-System

PART A. COMPREHENSIVE QUESTION

Case Study: Student Management System (STUDIA)

An educational institution maintains student information such as roll number, name, email address, department and academic year. Managing these records manually can lead to duplicated information, delayed updates and difficulty retrieving the latest student details. The institution requires a web-based Student Management System that stores records in a database and can be accessed through the internet.

Design and develop a database-driven Student Management System using React, Node.js, Express.js and MongoDB. Deploy the frontend and backend on suitable cloud platforms and make the application accessible through public web addresses.

The application is expected to support the following requirements:

- Store student details including roll number, full name, email, department and academic year; the implemented form also captures phone number, date of birth, gender and address.
- Create and manage a MongoDB database and students collection using an appropriate document structure.
- Perform CRUD operations: add a student, display student records, modify student details and delete a student.
- Provide search and filtering by student information, department and academic year.
- Use the MongoDB document identifier to locate a particular student during update and delete operations.
- Deploy the backend, frontend and database using cloud services. Explain hosting, public URLs and environment variables.
- Test the deployed application and confirm that data entered through the interface is stored in MongoDB and can be retrieved.
- Analyse possible reasons why a deployed application may successfully add records but fail to display updated records, and propose a systematic debugging procedure.

Problem Focus

The central case-study problem is a data-consistency and retrieval scenario: a create request may succeed while the interface continues to show stale or missing records. This report examines likely database-level and application-level causes and presents a repeatable debugging workflow. The submitted live project was also tested for add, view, edit and delete operations; the screenshots in Part C document the implemented interface and deployment evidence. The report does not claim that every hypothetical failure listed below occurred in the final deployed application.

PART B. WORKED SOLUTION

1. Analyse the Requirements

Requirement	STUDIA implementation
Student identity	Roll number is used as the human-readable student identifier; MongoDB _id identifies each stored document.
Student information	Name, rollNumber, email, department and year are core fields; the form additionally supports phone, dateOfBirth, gender and address.
Data storage	MongoDB Atlas database `test`, collection `students` on the existing `AURAFIT` cluster.
Application operations	React calls the Express API through Axios; Express uses Mongoose to read and modify MongoDB documents.
Search and filters	Client-side search and department/year filters are applied to the retrieved student list.
Deployment	Vercel hosts the frontend, Render hosts the backend, and MongoDB Atlas hosts the database.
Validation	Required student data and duplicate roll-number handling are addressed in the application/API workflow.

2. System Architecture

STUDIA follows a three-layer web architecture. The browser renders the React application. Axios sends HTTP requests to the Express API hosted on Render. The backend uses Mongoose to communicate with the MongoDB Atlas students collection. The frontend does not connect directly to MongoDB; database credentials remain on the backend.

```
User / Browser
      |
      v
React + Vite frontend (Vercel)
      |  HTTPS / Axios requests
      v
Node.js + Express REST API (Render)
      |  Mongoose
      v
MongoDB Atlas: test.students
```

3. Main Functional Modules

Module	Functionality
Dashboard	Displays the institutional overview and student/department counts.
Student directory	Lists student records and provides search and department/year filters.
Add student	Captures a new student's details and submits them to the backend.

View profile	Displays the selected student's stored information.
Edit student	Loads an existing record into an editable form and submits changes.
Delete student	Requests confirmation before deleting the selected record.

4. MongoDB Document Design

Each student is stored as a BSON document in the `students` collection. MongoDB automatically assigns an `_id` value. Mongoose timestamps are enabled in the observed document, so `createdAt` and `updatedAt` are available.

```
{
  "_id": "ObjectId(...)",
  "rollNumber": "A24126552098",
  "name": "Mohammed Almas",
  "email": "student@example.com",
  "department": "AI&DS",
  "year": 3,
  "createdAt": "ISODate(...)",
  "updatedAt": "ISODate(...)"
}
```

The example above shows the core fields observed in the stored record. The application form also collects phone number, date of birth, gender and address. Personal values are represented with placeholders in this schema example.

Field	Suggested type	Purpose
rollNumber	String	Institutional roll number; used for student lookup and uniqueness validation.
name	String	Student's full name.
email	String	Student contact email.
department	String	Academic department, for example AI&DS.
year	Number	Current academic year.
phone	String	Contact number.
dateOfBirth	Date / ISO date	Date of birth.
gender	String	Gender value selected in the form.
address	String	Student address.
createdAt / updatedAt	Date	Creation and last-update timestamps.

5. Database Setup and CRUD Queries

The project uses an existing MongoDB Atlas cluster named `AURAFIT`. Within that cluster, the database shown in Data Explorer is `test`, and the application collection is `students`. The cluster was reused; a separate new cluster was not created for this case study.

```
// Select the database
use test

// Read all student documents
db.students.find({})

// Find a student by roll number
db.students.findOne({ rollNumber: "A24126552098" })

// Update a student's department
db.students.updateOne(
  { rollNumber: "A24126552098" },
  { $set: { department: "AI&DS" } }
)

// Count stored students
db.students.countDocuments({})

// Delete a test record (verify the filter before execution)
db.students.deleteOne({ rollNumber: "TEST-ROLL-NUMBER" })
```

These are illustrative MongoDB Shell operations for the observed collection and field names. The live application performs its CRUD operations through the backend API rather than executing these shell commands from the browser.

6. Backend Development

The backend is implemented with Node.js, Express.js and Mongoose. It exposes REST endpoints under `/api/students`. The server reads the MongoDB connection string from the `MONGO_URI` environment variable and uses the hosting provider's assigned port when deployed.

Backend responsibilities

- Load environment variables from the backend environment.
- Configure Express middleware to parse JSON request bodies and support cross-origin requests from the frontend.
- Connect to MongoDB Atlas using Mongoose.
- Define student data through the Student model.
- Expose routes for retrieving, creating, updating and deleting student records.
- Return appropriate JSON responses and HTTP status codes for successful operations and errors.

REST API endpoint summary

Method	Endpoint	Purpose
GET	/api/students	Retrieve the student collection for the directory and dashboard.
POST	/api/students	Create a new student record.
PUT	/api/students/:id	Update a student using the MongoDB document ID.
DELETE	/api/students/:id	Delete a student using the MongoDB document ID.

The duplicate roll-number scenario is handled as a conflict: MongoDB duplicate-key error code 11000 is translated by the update route into HTTP 409 with a clear message. This prevents an edit from silently creating a duplicate institutional roll number.

Backend environment configuration

```
MONGO_URI=<MongoDB Atlas connection string>
PORT=<assigned by hosting platform or 5000>
```

The actual connection string is a secret and is stored in Render's Environment Variables settings. It must not be placed in frontend source code, committed to Git, or printed in the report.

7. Frontend Development

The frontend is a React application created with Vite. Axios is used through a shared API service module. The API base URL is configurable so the same frontend source can run locally or communicate with the deployed backend.

```
// frontend/src/services/api.js
baseURL:
  import.meta.env.VITE_API_URL ||
```

```
  "http://localhost:5000/api"
```

For production, Vercel is configured with `VITE_API_URL=https://studia-backend-orll.onrender.com/api`. The frontend uses React state to manage the retrieved student list, form values, loading/error messages, search text and selected filters.

Frontend workflow

1. On initial load, the application requests the student list from the backend.
2. The returned JSON data is stored in React state and rendered in the student directory.
3. Search and department/year filters derive a visible subset from the loaded list.
4. Add and edit forms submit their data to the corresponding API endpoint.
5. After a successful save, the form closes and the student list is fetched again so the latest server state is displayed.
6. Delete asks for confirmation, sends the delete request and refreshes the list after success.

8. Testing and Debugging Procedure

Case-study issue: records are added but updated records are not displayed

A successful create response does not by itself prove that the user interface has refreshed its data. The record may exist in MongoDB while the browser still displays an older React state, a cached response, or data from a different database/collection. Debugging should follow the request from the browser to the database and back.

Layer	Possible cause	How to verify / resolve
Frontend state	The list is not fetched after POST/PUT, or stale state is rendered.	Inspect React state and the save handler. After success, call the list-fetch function and replace the list with the returned data.
Frontend request	Wrong API base URL or request path.	Inspect the browser Network tab; verify request URL, method, payload and response.
Async flow	Refresh is called before the write completes.	Await the POST/PUT request, then await GET /api/students.
Response handling	UI assumes an outdated response shape or ignores errors.	Check response JSON and status; handle arrays and error objects correctly.
Backend route	PUT route does not update the expected document.	Verify route parameter, MongoDB _id, update filter and returned document.
Database selection	Backend connects to a different database or collection.	Check Render MONGO_URI and database selection; inspect the exact Atlas database and collection.
Schema/model	Field names or types differ between form, model and query.	Compare request JSON, Mongoose schema and stored document field names.
Validation / duplicate key	Update is rejected by validation or a unique index.	Inspect backend logs and HTTP status; display the returned validation/conflict message.
Caching / deployment	Old frontend build or stale browser cache is in use.	Confirm Vercel deployment is Ready, environment variables are correct, and reload the current production domain.
Connectivity / CORS	Frontend cannot reach the API or browser blocks the request.	Inspect console and Network errors; verify backend availability and allowed frontend origin.

Systematic debugging sequence

7. Reproduce the issue with one known test student and record the exact action, time and expected result.

8. Open browser Developer Tools → Network. Submit an update and inspect the PUT request URL, payload, status code and response body.
9. Check the backend logs on Render for route execution, validation errors, database errors and duplicate-key errors.
10. Call GET /api/students directly and compare its JSON response with the browser's displayed list.
11. Open MongoDB Atlas Data Explorer and inspect the `test.students` document. Confirm the correct record and `updatedAt` value.
12. Verify that the backend is connected to the intended Atlas cluster, database and collection; do not assume a successful connection means the intended database was selected.
13. Check the frontend API configuration (`VITE_API_URL`) and Render's environment variables. After changing a Vite environment variable, rebuild/redeploy the frontend.
14. Ensure the React save handler awaits the update request and then awaits a fresh GET request; verify that the resulting array is assigned to state.
15. Repeat the update, reload the page and confirm that the latest value remains visible. This verifies persistence rather than only a temporary UI change.

Test case matrix

Test ID	Test action	Expected result	Evidence / status
TC-01	Open dashboard	Dashboard loads and displays summary cards.	Dashboard screenshot captured.
TC-02	Open student directory	Stored student record is displayed.	Directory screenshot captured.
TC-03	Add a valid student	API stores record and refreshed list displays it.	Add form screenshot captured; CRUD workflow tested.
TC-04	View a student	Selected student's profile details are shown.	Profile screenshot captured.
TC-05	Edit student details	Updated record is saved and visible after refresh.	Edit form screenshot captured; update workflow tested.
TC-06	Delete a student	Confirmation is requested; confirmed deletion removes the record.	Confirmation screenshot captured; delete workflow tested.
TC-07	Duplicate roll number	Duplicate is rejected with a conflict message.	Backend handles duplicate-key error as HTTP 409.
TC-08	Inspect MongoDB	Student document is present in Atlas collection.	Atlas screenshot captured.
TC-09	Open deployed frontend	Vercel production deployment is Ready.	Vercel screenshot captured.
TC-10	Inspect backend	Render web service is Live.	Render screenshot

	deployment		captured.

Evidence note: the delete screenshot shows the confirmation dialog before the user confirms deletion. It is not presented as proof of the deletion result. The test workflow was separately confirmed in the running application.

9. Cloud Deployment

Deployment architecture

Layer	Platform	Configured project / role
Frontend	Vercel	Project `studia-frontend`; serves the React/Vite production build.
Backend	Render	Web service `studia-backend`; runs Node.js/Express API.
Database	MongoDB Atlas	Existing cluster `AURAFIT`, database `test`, collection `students`.

Deployment steps performed

16. Source code was maintained in the GitHub repository `mohammedalmas-csm98/FS-LAB`, under `CASE STUDY/Student-Management-System`.
17. The backend was deployed to Render as a Node.js Web Service. The backend root directory was set to `CASE STUDY/Student-Management-System/backend`, with `npm install` as the build command and `npm start` as the start command.
18. The MongoDB Atlas connection string was configured privately in Render as `MONGO_URI`; it was not committed to the repository.
19. The frontend was imported into Vercel from the same GitHub repository. Its root directory was set to `CASE STUDY/Student-Management-System/frontend` and its build output was `dist`.
20. The Vercel environment variable `VITE_API_URL` was set to the Render API base URL, ending in `/api`.
21. Both production deployments were checked: Vercel showed the frontend deployment as Ready, and Render showed the backend service as Live.
22. The deployed frontend was opened and the add, edit, view and delete workflows were tested. The student list and dashboard were confirmed to work.

Deployment URLs

Service	URL
STUDIA frontend	https://studia-frontend.vercel.app
STUDIA backend	https://studia-backend-orll.onrender.com
Student API endpoint	https://studia-backend-orll.onrender.com/api/students
GitHub repository	https://github.com/mohammedalmas-csm98/FS-LAB

Environment variable summary

Variable	Configured location	Purpose
MONGO_URI	Render backend service	Private MongoDB Atlas connection string.

PORT	Render runtime / backend	Port assigned by the hosting platform; local fallback is 5000.
VITE_API_URL	Vercel frontend project	Public backend API base URL used by the frontend.

Security and hosting considerations

- Never expose MONGO_URI in React code or a VITE_ variable. Vite variables are bundled into client-side assets and are not secrets.
- Keep `.env` files out of Git. The project uses ignore rules to prevent local secrets and dependency folders from being committed.
- Use a strong database credential and restrict Atlas network access according to the hosting and security requirements.
- Render free services may spin down after inactivity; the first request after an idle period can take longer to respond.
- Use HTTPS public URLs and verify frontend-to-backend connectivity after every environment-variable or deployment change.

PART C. OUTPUT SCREENSHOTS AND EVIDENCE

The following figures are captured from the implemented STUDIA application and its deployment/database consoles. They are included as execution evidence for the submitted project.

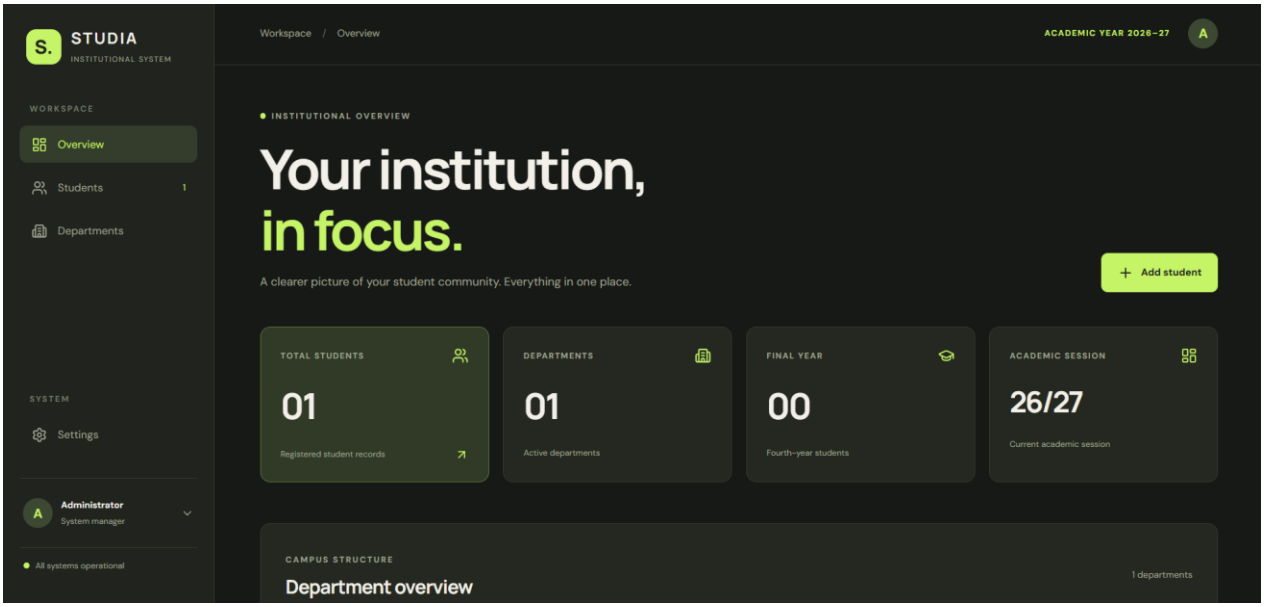


Figure 1. STUDIA dashboard showing the institutional overview and current student summary.

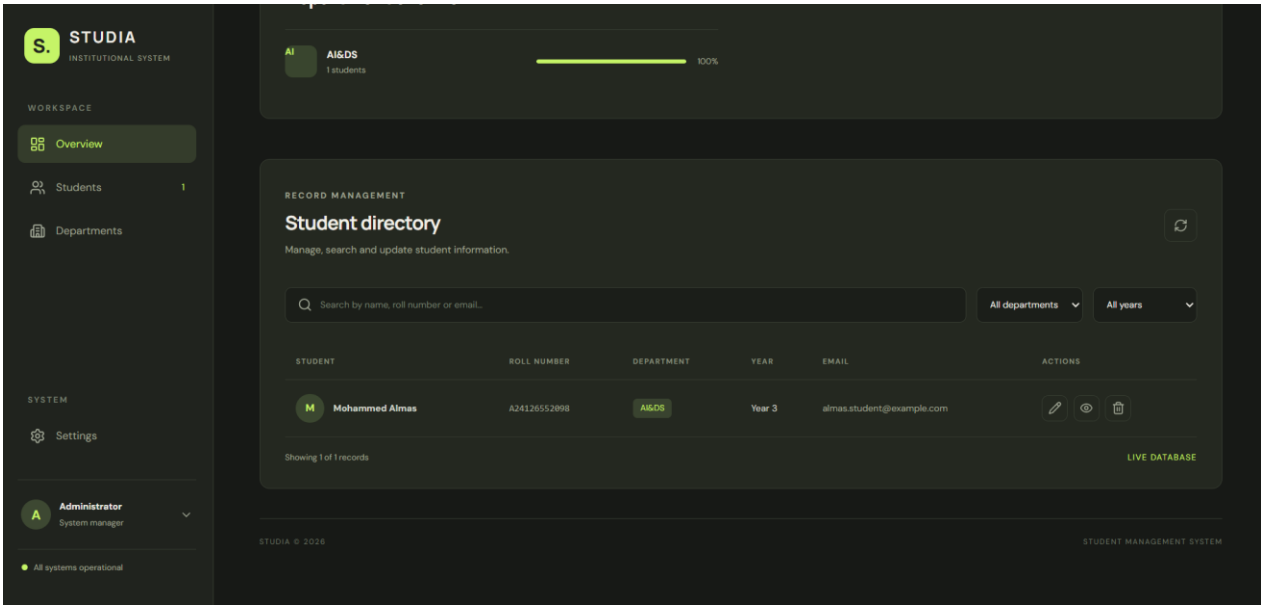


Figure 2. Student directory showing the stored student record and available list actions.

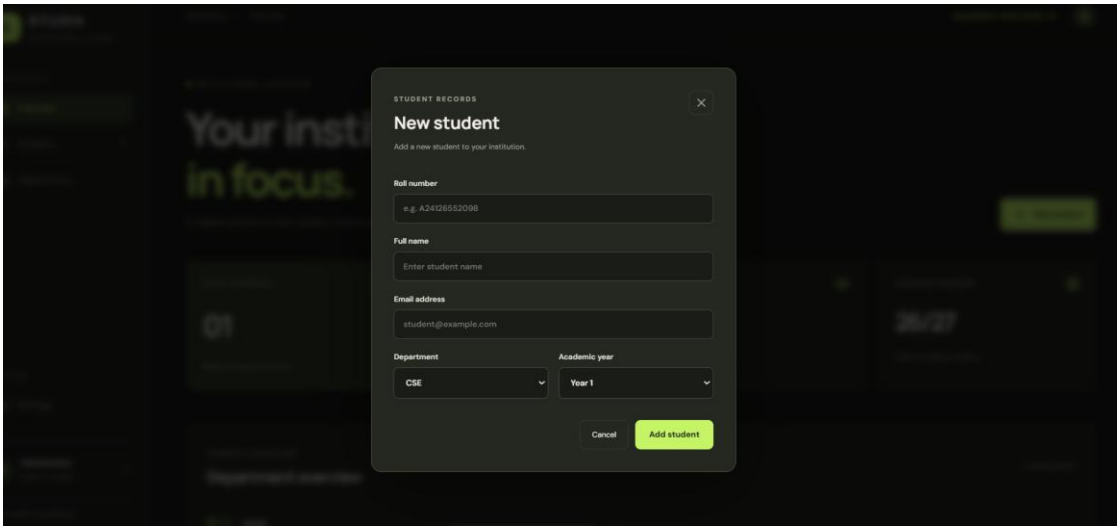


Figure 3. Add Student form with academic and personal information fields.

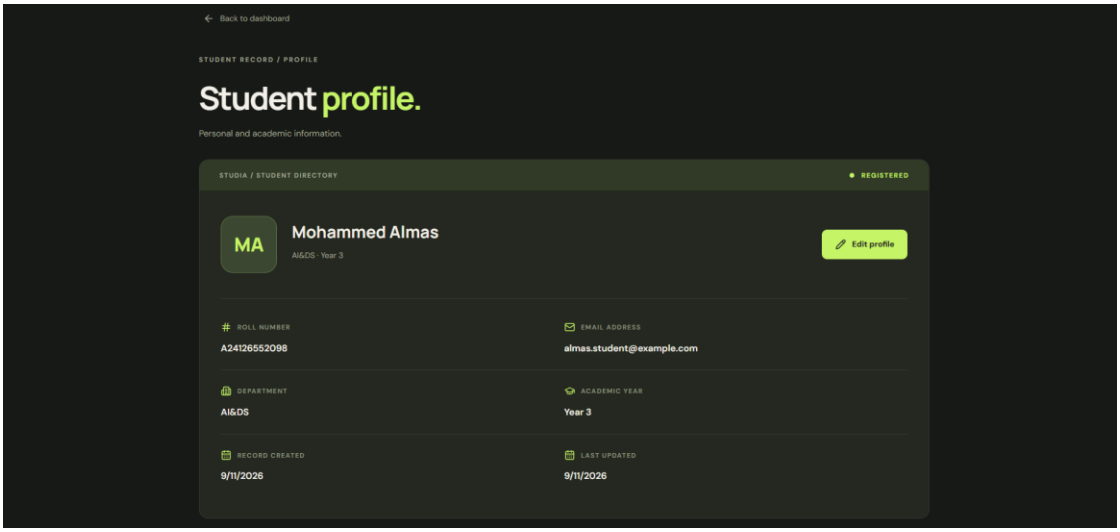


Figure 4. Student profile page displaying the selected student's record.

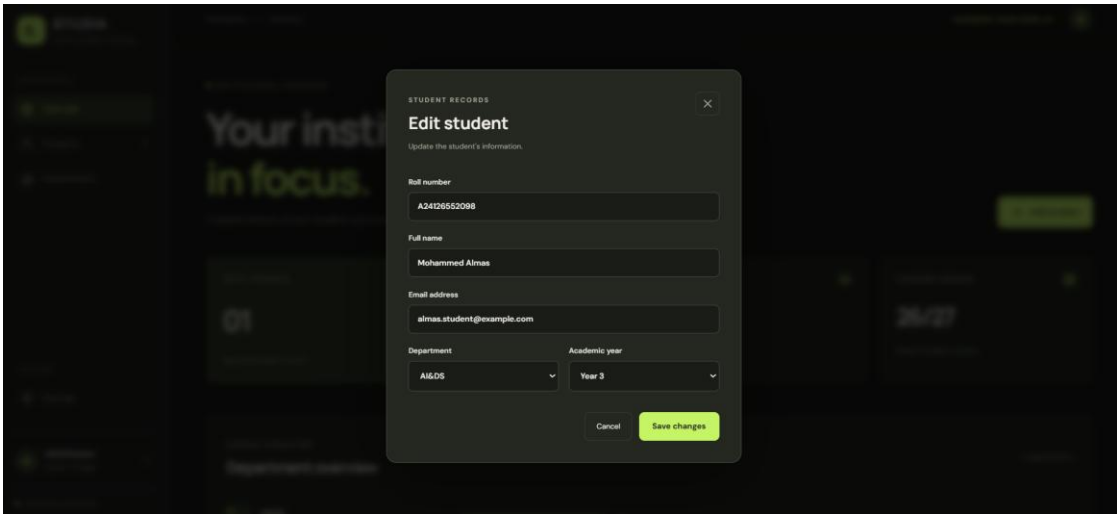


Figure 5. Edit Student form populated with the selected student's existing values.

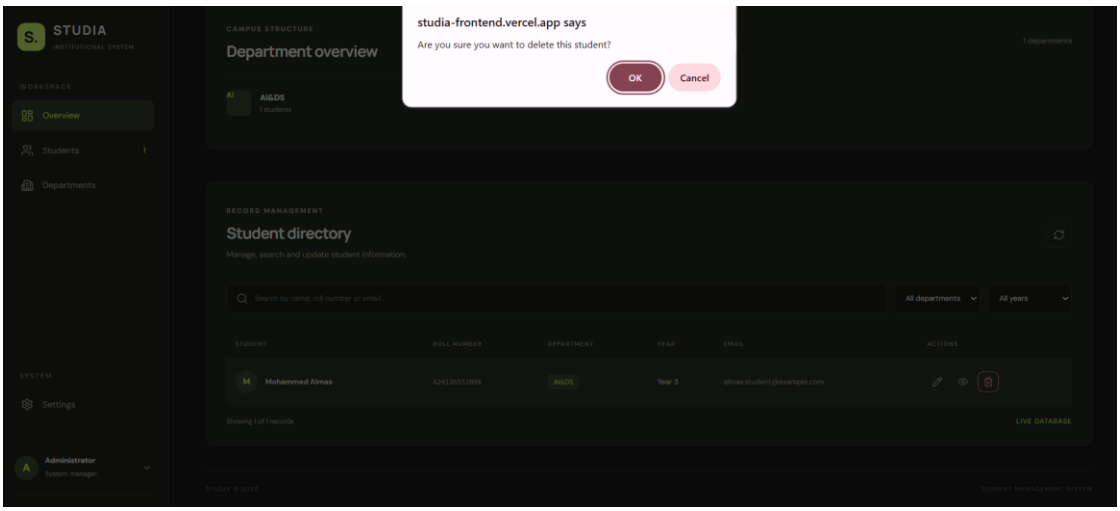


Figure 6. Delete confirmation dialog displayed before a deletion is confirmed.

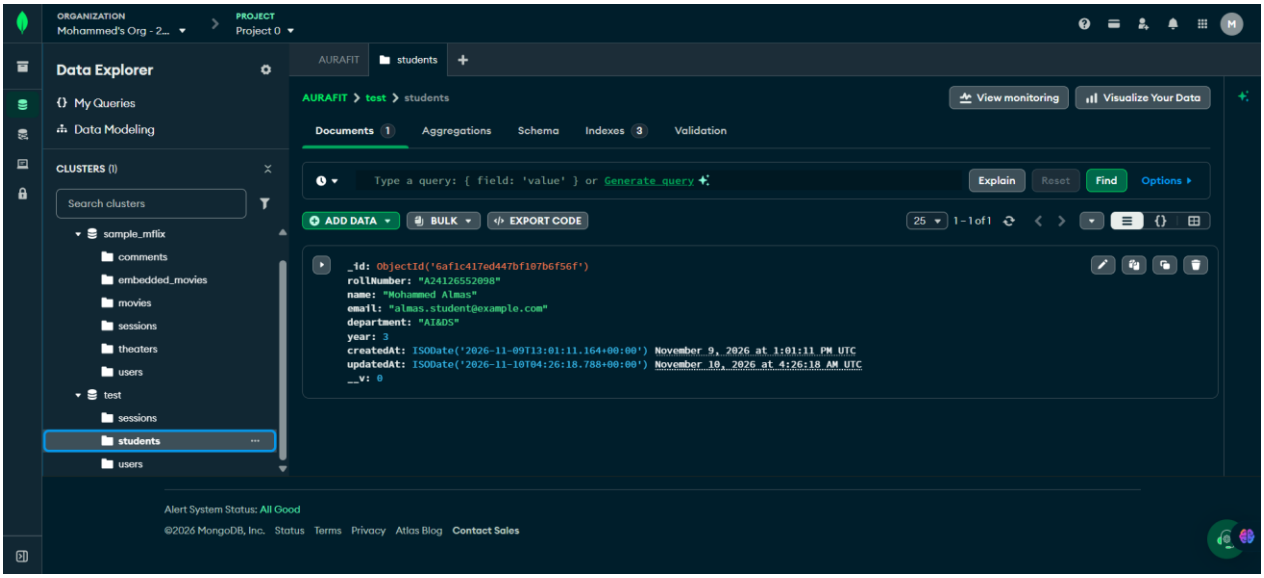


Figure 7. MongoDB Atlas Data Explorer showing the `test.students` collection and one stored student document.

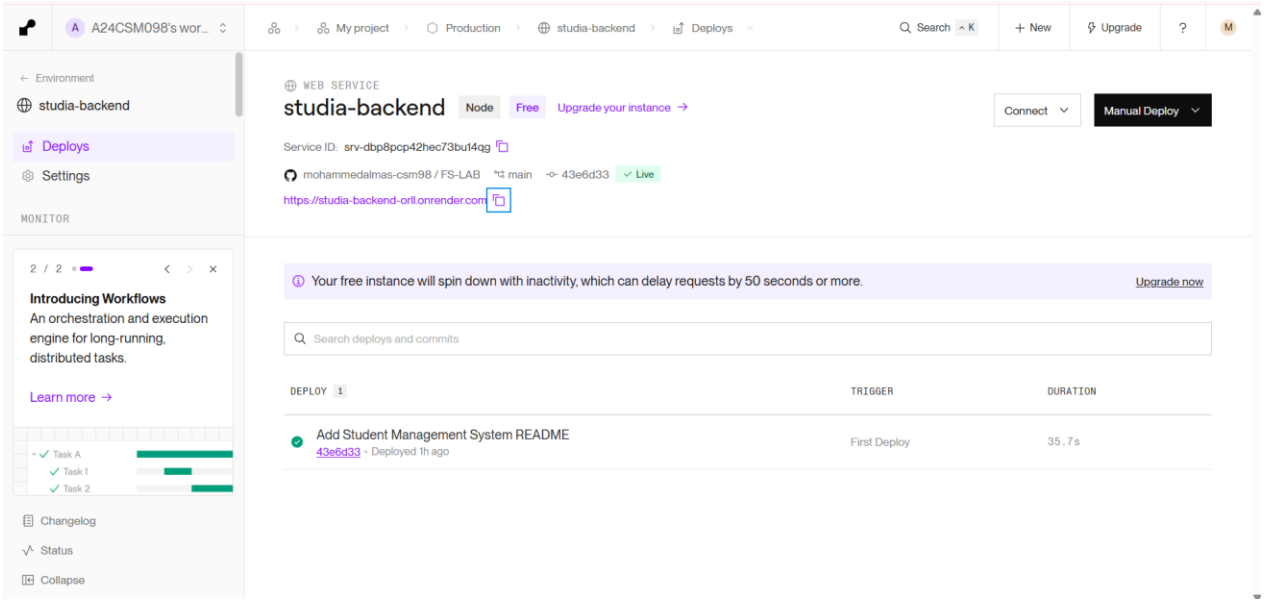


Figure 8. Render dashboard showing the `studia-backend` Node.js service with Live status.

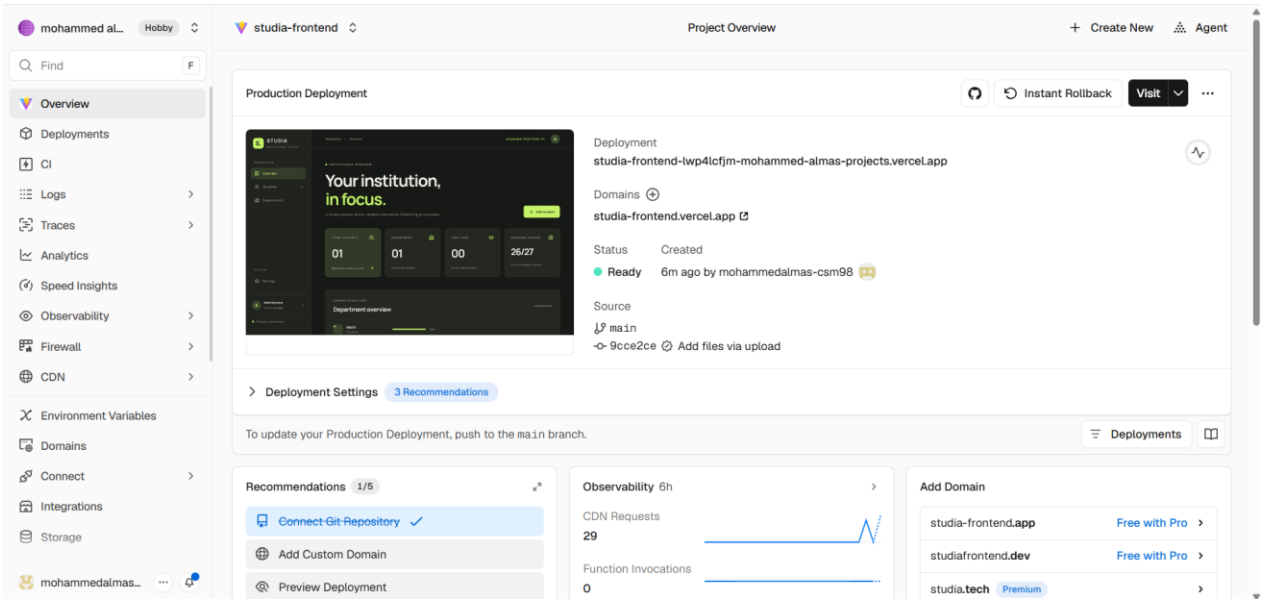


Figure 9. Vercel project overview showing the `studia-frontend` production deployment with Ready status.

Evidence interpretation

The frontend screenshots demonstrate the user-facing workflows. The MongoDB Atlas screenshot demonstrates that a student document is persisted in the configured collection. The Render and Vercel screenshots demonstrate that the backend and frontend deployments reached their respective Live and Ready states. Together, these pieces of evidence support the deployed three-layer architecture.

10. Expected Outcome and Conclusion

Expected outcome

- A public STUDIA frontend that can be opened through the Vercel production URL.
- A deployed Express backend that communicates with MongoDB Atlas.
- Persistent student records that can be created, retrieved, updated and deleted through the application.
- A student directory with search and department/year filtering, together with an individual profile view.
- Deployment evidence from Vercel, Render and MongoDB Atlas.

Conclusion

STUDIA demonstrates the development and deployment of a full-stack Student Management System using React, Node.js, Express.js and MongoDB. The application provides a dashboard, student directory, add/edit forms, profile viewing and deletion confirmation. MongoDB Atlas stores the student records, while the Express API acts as the controlled interface between the database and the React frontend.

The case-study debugging scenario highlights that a successful database write and a refreshed user interface are separate parts of a complete workflow. Reliable verification requires checking the browser request, backend route and logs, database document, API read response and React state refresh. The deployed project was tested for its core CRUD workflows, and the screenshots document the application and hosting configuration.

Limitations and future enhancements

- Add role-based authentication for administrators, faculty and students.
- Introduce pagination and server-side filtering for large student datasets.
- Add export to CSV/PDF and printable student reports.
- Add audit logs to track who changed a student record and when.
- Add automated API and end-to-end tests to support future releases.
- Add stronger production monitoring, backups and restricted database network access.

References

- Course reference material: Sample Case Study for Reference, Unit V – MongoDB, Database Operations and Cloud Deployment.
- MongoDB documentation: https://www.mongodb.com/docs/
- Express documentation: https://expressjs.com/
- React documentation: https://react.dev/
- Vite documentation: https://vite.dev/
- Render documentation: https://render.com/docs
- Vercel documentation: https://vercel.com/docs